#	Layers	Heads	Attention-only	Optimizer	Note
1	2	8	True	Adam	
2	2	8	True	AdamW	
3	2	8	False	Adam	
4	2	8	False	AdamW	
5	2	16	True	Adam	
6	2	16	True	AdamW	
7	2	16	False	Adam	
8	2	16	False	AdamW	
9	4	8	True	Adam	
10	4	8	True	AdamW	
11	4	8	False	Adam	
12	4	8	False	AdamW	
13	4	16	True	Adam	
14	4	16	True	AdamW	
15	4	16	False	Adam	
16	4	16	False	AdamW	
17	8	8	True	Adam	
18	8	8	True	AdamW	
19	8	8	False	Adam	
20	8	8	False	AdamW	
21	8	16	True	Adam	
22	8	16	True	AdamW	
23	8	16	False	Adam	
24	8	16	False	AdamW	
25	12	12	False	Adam	* GPT2-small with Adam
26	12	12	False	AdamW	* GPT2-small with AdamW

Table 1. Details of 26 architectures

A.3. Training Configuration

Unless noted otherwise, training choices are held at standard defaults to isolate architectural effects; **Adam** (constant learning rate) is the baseline optimizer, with a mirror set trained using **AdamW** (decoupled weight decay). Training configuration templates are shown in (Listing 4, 5, 6 and 7). We define these templates using **TransformerLens HookedTransformerTrainConfig**; any hyperparameters not listed use the default values. More details can be found in training code provided in supplementary material.

A.4. Prompts Sets & Generation

We used two prompts resources:

- **Primary set (100 prompts)**: Used for all experiments except § 4.4.2. This set was synthesized using ChatGPT (GPT-5.2 Thinking) model; provided at § A.4.
- **Length-sweep set (6 x 20 prompts)**: Six subsets derived from the same 20 base prompts by truncation, yielding target length of 5, 10, 20, 30, 40, and 50 tokens per prompt. This set was used for § 4.4.2 and is shown in § A.4.

Actual prompt sets are provided within supplementary material.

Instruction prompt to generate Primary set of 100 prompts

```
You are generating an evaluation set for a **non{instruction-tuned, GPT-2-small{style**
language model.  Produce **exactly 100** diverse **text prompts (prefixes)** that a
model can naturally continue.
CRITICAL OUTPUT REQUIREMENTS (to avoid Python SyntaxError):  - Output format:  **a
single valid Python list literal** of **exactly 100** elements, e.g.  ["...", "...",
```

```
...]  - Use **ONLY double-quoted Python strings** for every prompt.  - Do **NOT** use
triple quotes (no """ and no ’’’).  - Do **NOT** include any unescaped double quotes
inside strings.  If a prompt would include quotes, use **single quotes** in the text (’)
or escape as ¨ Escape special characters when needed:  backslash as
and newlines as
n.  - Do **NOT** wrap the output in markdown fences, and do **NOT** include any
explanation text before or after the list.
PROMPT CONTENT REQUIREMENTS: - Each prompt is **15{80 tokens** (roughly 1{4 sentences)
and is an **incomplete prefix** (it should not feel finished).  - Prompts must be
**random, varied, and non-duplicative** (no near-copies).  - Avoid requiring external
knowledge; questions should be answerable from context or be \continue the story" type.
- Keep content safe:  no hate, sexual content, self-harm, instructions for wrongdoing,
or personal data.
DIVERSITY REQUIREMENTS (spread across the 100 prompts):  - Genres:  realistic, fantasy,
sci-fi, mystery, romance (PG), humor, horror (mild), slice-of-life.  - Formats:
narration, dialogue, interview transcript, news-style paragraph, diary entry, recipe
step-by-step, meeting notes, customer support chat, legal/contract-ish clause,
scientific abstract-ish intro.  - Structured text:  bullet list starts, numbered list
starts, parentheses/brackets, quoted speech using single quotes, markdown-like headers,
a short poem start, a letter/email opening, a \Q: ...  A:" snippet.  - Lightweight
reasoning:  simple arithmetic embedded in text, basic logic constraints, pattern
completion (e.g., \A, B, C, ..."), cause-effect setups.  - Code-ish prefixes:  include a
few prompts that begin a function or config snippet (Python/JSON/pseudocode) but do not
require correctness|just naturally continuable.  Ensure all code-like lines are inside
the same double-quoted string and use
n for line breaks.
FINAL CHECK BEFORE YOU OUTPUT: - Count elements:  exactly 100 strings.  - Ensure the
entire output can be pasted into Python without errors.
Now output only the Python list literal of 100 prompt strings.
```

Instruction prompt to generate ”Length-sweep sets” with varying prompt length

```
You are generating an evaluation set for a **non{instruction-tuned, GPT-2-small{style**
language model.
GOAL: Create **6 length-controlled sets** of prompts by sweeping the hyperparameter **i
∈ [5, 10, 20, 30, 40, 50]**.  Each set contains **exactly 20 prompts**.  The prompts
should be **random, varied, and naturally continuable** prefixes.
CRITICAL LENGTH RULE (how to count \tokens"):  - Define \token" as a **space-separated
word** using this exact rule:  ‘tokens = text.split(" ")‘.  - Therefore, each prompt
must contain **exactly i tokens** meaning **exactly i space-separated chunks**.  - Use
**single spaces only** between tokens.  No leading/trailing spaces.
CONSISTENCY RULE ACROSS SETS: - The 20 prompts represent 20 \base prompts" (same
identity across lengths).  - For each base prompt k (1..20), create a **master version
of exactly 50 tokens**.  - Then derive shorter versions by **truncation**:  - The i=5
version is the **first 5 tokens** of the master.  - The i=10 version is the **first 10
tokens** of the master.  - Similarly for i=20,30,40,50.  - This ensures:  **only the
number of tokens changes**, and content is otherwise identical across sets (shorter is a
prefix of longer).
CONTENT REQUIREMENTS (for the 20 master prompts):  - Each master prompt must be **50
tokens exactly** (per the split rule).  - Prompts must be **incomplete prefixes** (do
not feel finished; okay to end mid-sentence).  - Prompts must be **diverse** across the
20 items:  - Genres/voices:  realistic, fantasy, sci-fi, mystery, mild horror, humor,
slice-of-life.  - Formats:  dialogue, diary entry, news-style lead, meeting notes,
customer support chat, recipe steps, faux scientific abstract opening, legal-ish clause
opening, Q/A snippet, poetic start.  - Include a few with lightweight structure:  a
short numbered list or bullet-like tokens, parentheses, brackets.  - Include a few
with lightweight reasoning cues (simple arithmetic or constraints) but not requiring
correctness.  - Include 2{3 code-ish natural prefixes (e.g., \def", \", \if") BUT still
plain text; no newlines.
SAFETY REQUIREMENTS: - No hate, sexual content, self-harm, personal data, or
instructions for wrongdoing.
OUTPUT FORMAT REQUIREMENTS (to avoid Python SyntaxError):  - Output **ONLY** a single
valid **Python dict literal** with exactly these integer keys:  5, 10, 20, 30, 40, 50 -
Each key maps to a Python list of **exactly 20** strings.  - Use **ONLY double-quoted
```

```
Python strings** for every prompt.  - Do **NOT** use triple quotes (no """ and no ''').
- Do **NOT** include any unescaped double quotes inside strings (avoid double-quote
characters entirely).  - No newlines inside strings.  - Do not add comments, markdown,
or any text before/after the dict.
FINAL CHECK BEFORE OUTPUT: - For every i, every string has exactly i tokens by the
split rule.  - For each k in 1..20, dict[5][k] equals the first 5 tokens of dict[50][k],
dict[10][k] equals first 10 tokens of dict[50][k], etc.  - Exactly 6 keys and exactly 20
prompts per key.
Now output the Python dict literal.
```

A.5. Stability of head h_i w.r.t other individual pair refit

Figure 12 visualizes per-head stability with respect to 50 individual pair refits for an 8-layer, 8-head MLP model. It shows a series of heatmaps, one per layer, for all heads in the anchor refit (seed = 1). The x-axis indexes pair refits, so each cell reports the stability of a single head against a single pair refit. See Figure 12.

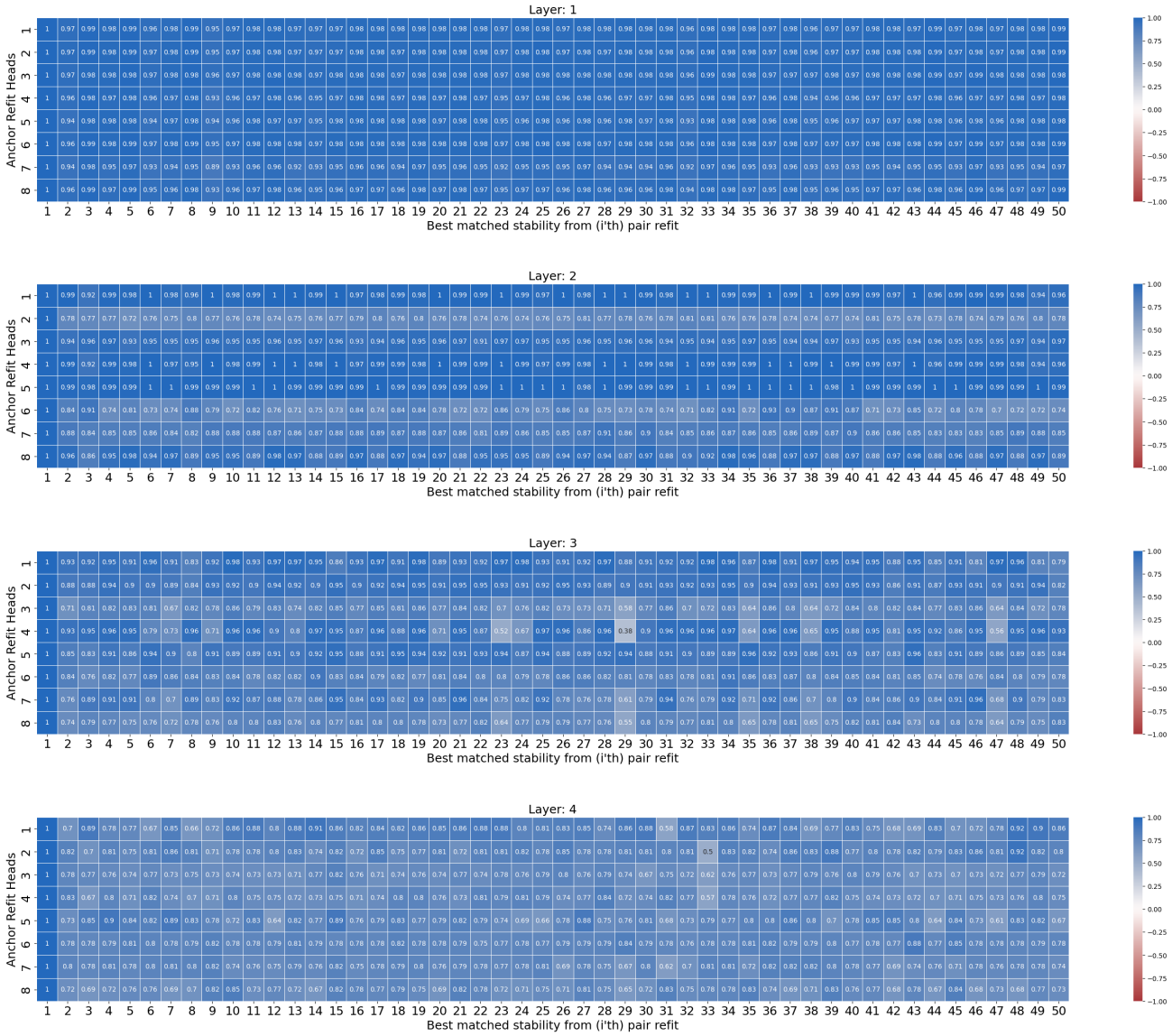

Figure 12. Residual stream is relatively more stable than its corresponding attention heads.